

FRAGMENTACIÓN HORIZONTAL EN LA TABLA DE CUSTOMER

José Luis Martínez De la rosa

Luis Ángel Díaz Domínguez

MINITERMINOS

- m1 = (TerritoryID = 1 AND StoreID IS NOT NULL)
- m2 = (TerritoryID = 2 AND StoreID IS NOT NULL)
- m3 = (TerritoryID = 3 AND StoreID IS NOT NULL)
- m4 = (TerritoryID = 4 AND StoreID IS NOT NULL)
- m5 = (TerritoryID >= 5 OR StoreID IS NULL OR TerritoryID IS NULL)

Los minitérminos están divididos en territoryID. Los primeros 5 ID corresponden a las 5 zonas en EU.

Como después se ve en el código secciono 5 tablas por 3 ID'S. Posteriormente explico la razón y el minisistema que puede subsistir por tienda.

Los tamaños de cada tabla son los siguientes:

NODO 1 – 3,520

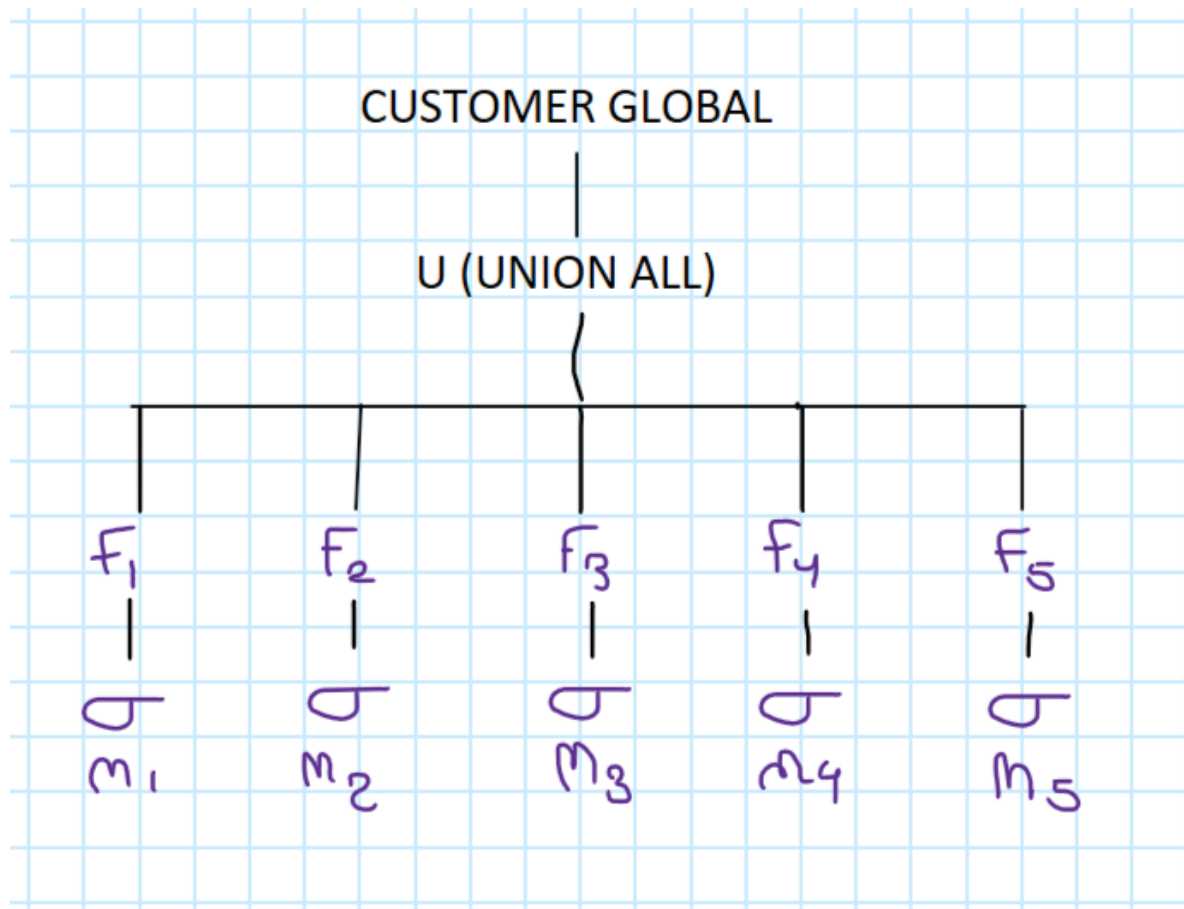
NODO 2 - 113

NODO 3 – 132

NODO 4 – 4,696

NODO 5 – 176

GRAFO RELACIONAL



-- NODO 1: Clientes Rango Northwest

```
CREATE TABLE dbo.Customer_Nodo1 (
```

```
    CustomerID INT NOT NULL,
```

```
    StoreID INT NULL,
```

```
    TerritoryID INT NULL,
```

```
    CONSTRAINT PK_Customer_Nodo1 PRIMARY KEY CLUSTERED (CustomerID),
```

```
    CONSTRAINT CK_Customer_Nodo1_Rango CHECK (TerritoryID = 1)
```

```
);
```

-- NODO 2: Clientes Rango Northeast

```
CREATE TABLE dbo.Customer_Nodo2 (  
    CustomerID INT NOT NULL,  
    StoreID INT NULL,  
    TerritoryID INT NULL,  
    CONSTRAINT PK_Customer_Nodo2 PRIMARY KEY CLUSTERED (CustomerID),  
    CONSTRAINT CK_Customer_Nodo2_Rango CHECK (TerritoryID = 2)  
);
```

-- NODO 3: Clientes Rango Central

```
CREATE TABLE dbo.Customer_Nodo3 (  
    CustomerID INT NOT NULL,  
    StoreID INT NULL,  
    TerritoryID INT NULL,  
    CONSTRAINT PK_Customer_Nodo3 PRIMARY KEY CLUSTERED (CustomerID),  
    CONSTRAINT CK_Customer_Nodo3_Rango CHECK (TerritoryID = 3)  
);
```

-- NODO 4: Clientes Rango Southwest

```
CREATE TABLE dbo.Customer_Nodo4 (  
    CustomerID INT NOT NULL,  
    StoreID INT NULL,  
    TerritoryID INT NULL,  
    CONSTRAINT PK_Customer_Nodo4 PRIMARY KEY CLUSTERED (CustomerID),  
    CONSTRAINT CK_Customer_Nodo4_Rango CHECK (TerritoryID = 4)  
);
```

-- NODO 5: Clientes Rango Southwest

```
CREATE TABLE dbo.Customer_Nodo5 (  
    CustomerID INT NOT NULL,  
    StoreID INT NULL,  
    TerritoryID INT NULL,  
    CONSTRAINT PK_Customer_Nodo5 PRIMARY KEY CLUSTERED (CustomerID),  
    CONSTRAINT CK_Customer_Nodo5_Rango CHECK (TerritoryID = 5)  
);
```

-- Poblado del Nodo 1

```
INSERT INTO dbo.Customer_Nodo1 (CustomerID, StoreID, TerritoryID)  
SELECT CustomerID, StoreID, TerritoryID  
FROM Sales.Customer  
WHERE TerritoryID = 1;
```

-- Poblado del Nodo 2

```
INSERT INTO dbo.Customer_Nodo2 (CustomerID, StoreID, TerritoryID)  
SELECT CustomerID, StoreID, TerritoryID  
FROM Sales.Customer
```

```

WHERE TerritoryID = 2;

-- Poblado del Nodo 3
INSERT INTO dbo.Customer_Nodo3 (CustomerID, StoreID, TerritoryID)
SELECT CustomerID, StoreID, TerritoryID
FROM Sales.Customer
WHERE TerritoryID = 3;

-- Poblado del Nodo 4
INSERT INTO dbo.Customer_Nodo4 (CustomerID, StoreID, TerritoryID)
SELECT CustomerID, StoreID, TerritoryID
FROM Sales.Customer
WHERE TerritoryID = 4;

-- Poblado del Nodo 5
INSERT INTO dbo.Customer_Nodo5 (CustomerID, StoreID, TerritoryID)
SELECT CustomerID, StoreID, TerritoryID
FROM Sales.Customer
WHERE TerritoryID = 5;
GO

CREATE VIEW dbo.Customer_Global AS
SELECT CustomerID, StoreID, TerritoryID FROM dbo.Customer_Nodo1
UNION ALL
SELECT CustomerID, StoreID, TerritoryID FROM dbo.Customer_Nodo2
UNION ALL
SELECT CustomerID, StoreID, TerritoryID FROM dbo.Customer_Nodo3
UNION ALL
SELECT CustomerID, StoreID, TerritoryID FROM dbo.Customer_Nodo4
UNION ALL
SELECT CustomerID, StoreID, TerritoryID FROM dbo.Customer_Nodo5;
GO

-- Consulta sobre la vista en su propio bloque
SELECT * FROM dbo.Customer_Global;
GO

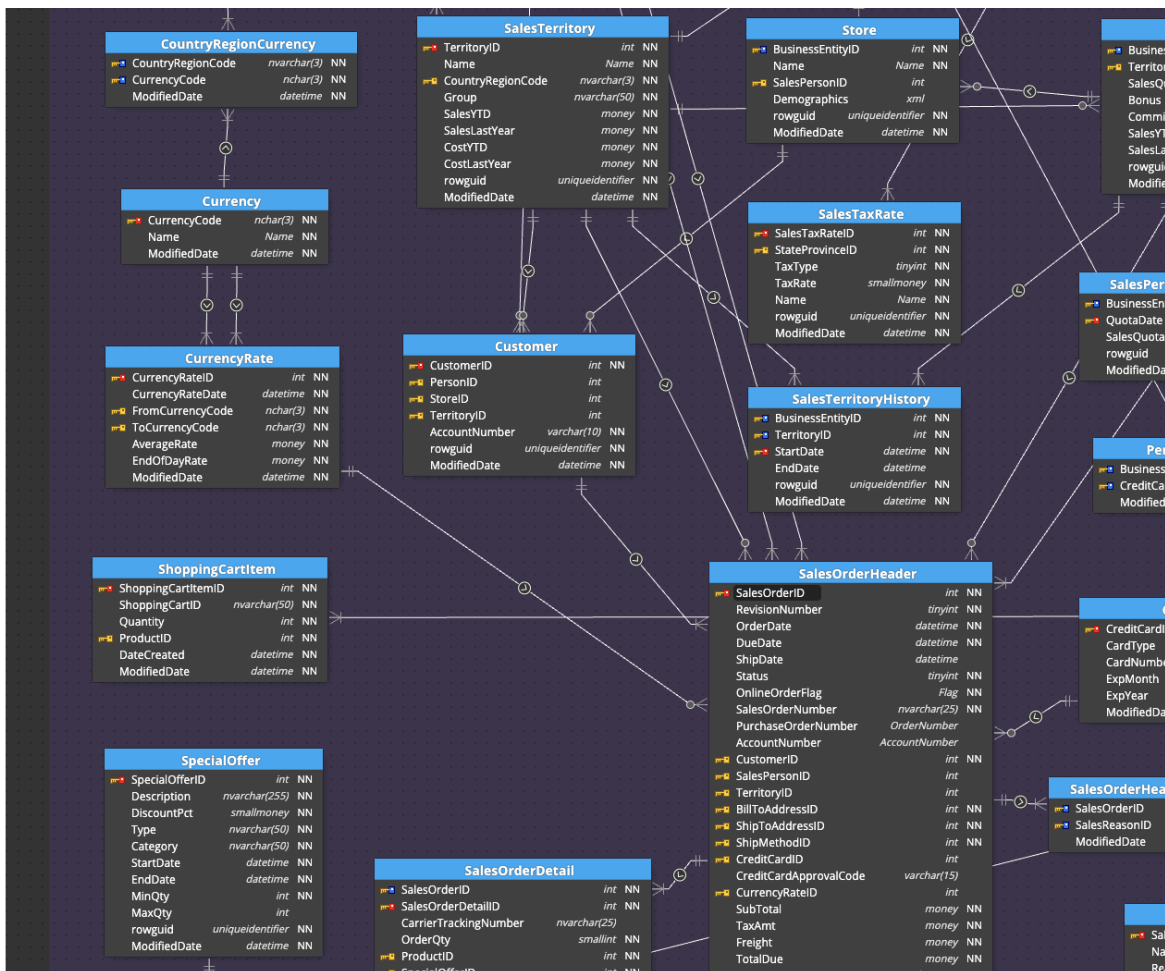
```

Las tablas de negocio relacionadas directamente son SalesTerritory, Store, SalesOrderHeader.

Y de estas salen a consultas más precisas y requeridas en las ventas diarias como SalesPerson, Person y bueno, son tablas de esquemas diferentes, uno de person y otro del mismo Sales. Aunque igual sale a producción por la tabla de productListPriceHistory que conecta directo a la tabla de Product.

Pero el corazón de esta segmentación de mercado está dentro del mismo esquema al que pertenece, y justo por eso elegimos este ejercicio, pues el corazón de la empresa

son sus ventas y por tal motivo es el esquema más importante y complejo de casi cualquier empresa y a partir de este se arma todo el TTL.



Ese una parte del esquema de sales y bueno todas las tablas están realmente conectadas y en específico la información de id de cliente y de tienda son necesarios para monitorear las ventas realizadas y que vallas conforme a las reglas de negocio de las cuales varias de estas van saliendo durante nuevos análisis hechos dentro de la empresa.

Por ejemplo:

```
SELECT soh.SalesOrderID, soh.SubTotal, soh.OnlineOrderFlag FROM Sales.SalesOrderHeader soh
INNER JOIN dbo.Customer_Nodo4 N4 ON N4.CustomerID = soh.CustomerID
```

En donde ya estamos ocupando la tabla seccionada por territorio y así evitamos recorrer todos los registros, de todas las demás regiones, optimizando así los recursos de la base.

Al igual pase solo tres atributos con llaves importantes en la tabla porque los demás eran puramente administrativos y esto iba a consumir recursos de mas de esta forma se cumple el COM_MIN y si nos fijamos al final del código podemos observar que las 5 tablas se pudieron volver a unificar en una sola con el UNION ALL.